

Plexus Discovery

Progress note — written to be read, not decoded

31 July 2026

Plain English throughout. Organised by phase, because phases are where we stop and you decide.

1. What we are doing — two things at once

Track A — build the method. An automated loop that searches for biological *mechanisms* rather than parameter values. It proposes a change to the model’s structure (“remove the pushing force”, “add oriented cell division”), writes down what it expects *before* running, runs the simulation, measures, and records whether it was right. The discipline that makes it worth building: a change of numbers can never masquerade as a new idea.

Track B — reproduce Okuda’s result, as the proof that Track A works. Okuda et al. (2018) grew a hollow ball of cells that spontaneously forms tubes, undulations and branches, driven by a chemical pattern on its own surface. We are rebuilding that here. If the loop finds the mechanism largely by itself, the method is demonstrated. *The figure is not a side quest — it is the evidence.*

The two tracks feed each other: the reproduction is the honest test of the method, and the method is what makes the reproduction more than hand-tuning.

2. Where we actually are

Track A is further along than Track B, and every number taken before 31 July was measured with at least one instrument since proved wrong. So the register of results is empty on purpose, and the work right now is rebuilding the instruments rather than running experiments.

3. What we believe about the biology

Nothing *measured*. But we now write down what we assume.

The register of *results* is still empty, and should stay empty until a batch has been taken with instruments that work. Every number we have was measured with at least one broken ruler in the path.

What changed on 31 July is a different thing: the register of **assumptions** is no longer empty. Eleven basics about cell tissue are written down in `discovery_okuda/PREMISES.md` — not a literature review, but the minimum a person needs in order to look at one of our runs and say “that cannot be right”. Each is phrased so that a computer can check it against a run, and each one now does (Phase 1).

The distinction matters. A result is something we found out. A premise is something we are *assuming*, stated plainly enough that it can be caught being wrong. Writing the second kind down is not a claim about biology — it is a way of making our own mistakes visible.

4. Two questions instead of one number

Success is not a threshold a single run clears. It is two questions, and they are asked separately because they fail separately.

1. What can we reproduce?

Each of the paper’s morphologies is a row, scored qualitatively: *not attempted* / *attempted and failed* / *partial* / *reproduced*. **Today 0 of 4.** Budding is tracked too — our substrate produces it and no criterion ever mentioned it.

2. What do we know?

Already measured, and previously subordinate to the tube gate: map coverage **27%** (3 of 11 cells can state a verdict) — and all of it is routing; solo operator effects are **0 of 8**. Nine claims resolved, ten open, surprise rate **0.375**.

5. What is genuinely working

Four things have earned their keep, and they are all about catching ourselves rather than about tissue.

Predictions are written before runs, and wrong ones stay wrong. The findings register now holds **six standing claims and six retracted**, each retraction carrying the reason it was believed in the first place. Nothing is quietly edited away.

Instruments are certified against a known answer before they are used. This already cost one of my own: a wavelength measure failed its own calibration — the ratio it depended on came out 1.37, 1.79, 1.49 instead of a constant — and it was demoted to a labelled diagnostic instead of being shipped. A metric that fails certification does not get to pass.

The premise gate has caught something on its own. A smoke run was refused before any result was read, because the chemistry had been extinguished by growth dilution. That is the first time anything in this project stopped a bad run without a human looking at a picture.

The loop refuses to guess. Asked to score 32 results against predictions written in a unit it did not recognise, it declined 32 times rather than inventing 32 confirmations.

The standing caveat. Every defect found so far — and there have been more than twenty —

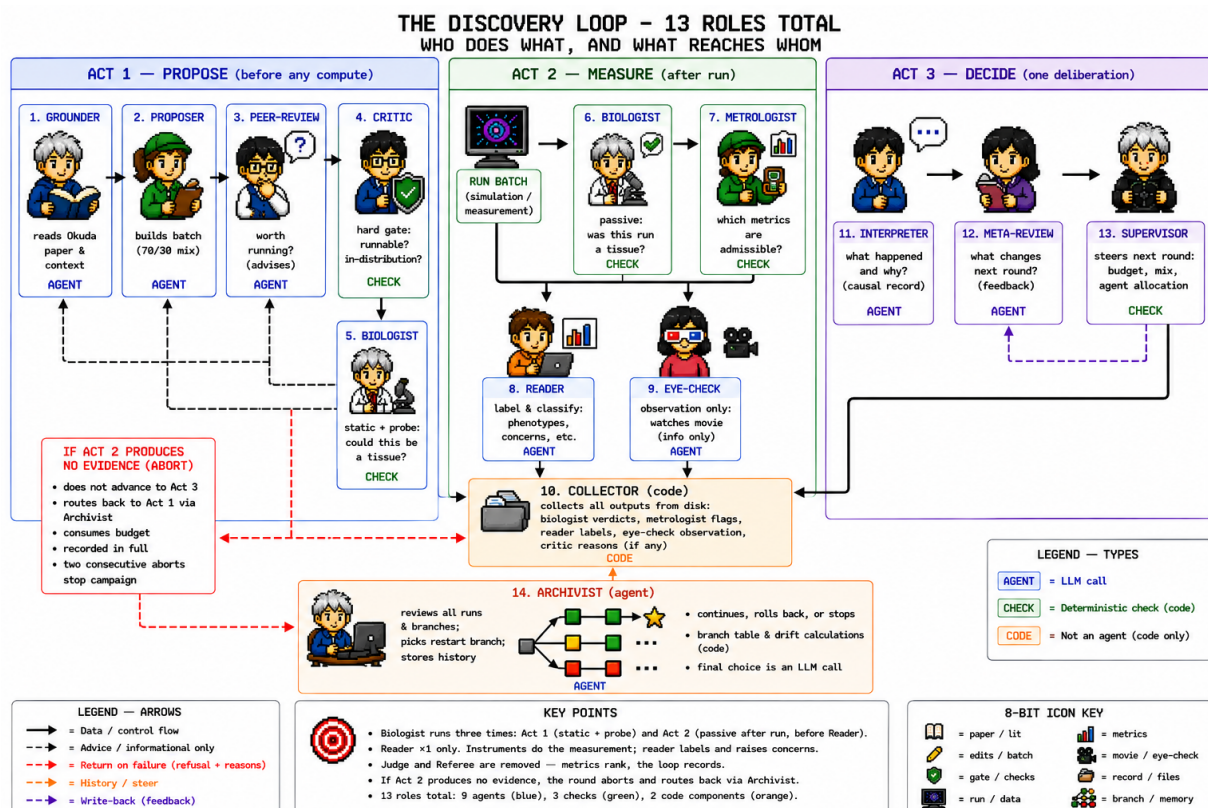
was found by Cedric looking at a picture. The gate above is the first thing built to change that, and it has one catch to its name. That number is the honest measure of Track A.

6. The team of agents

The roster was rebuilt on 1 August, after an external review, and it is now written down. `ROLES.md` is the source of truth — the same discipline `PREMISES.md` has for the biology — and the code is checked against it in both directions by `roles.py --check`. The campaign had reached **sixteen roles that nobody could account for**: three had never been called once, and the Biologist ran on every single run while talking to nobody. None of that was visible, because the roster lived in four places at once — the code, a table in this note, a figure drawn by hand, and everybody’s memory. Four descriptions of one thing are three descriptions that are wrong.

The cause was a mismatch, and it is worth stating plainly. We took *Co-Scientist’s* roster and put it on *Robin’s* problem. Co-Scientist runs a tournament of hypotheses because **it performs no experiments** — its evidence is literature, so proposals can only be debated. Robin has no tournament and no critic, because it *measures*. We measure. Where a number exists, a tournament is a worse ranker than the number, and our Judge and Referee were duly called **zero times** in the live run: there was nothing for them to do. The same gap explains the Biologist — Co-Scientist has no slot for “*the experiment was invalid*” because it has no experiments, so we invented the role and the roster had nobody for it to talk to.

The rule that now governs the roster: a role exists only where it adds information that is not already available. Judgement is slow, expensive and unaccountable; where a question has a deterministic answer, code answers it. **Thirteen roles: eight agents and five deterministic checks**, one of which — the Collector — was an agent’s job and should never have been. The roster went **16** → **13**: Judge, Referee, Evolution and the duplicate check out; the **Archivist** and the **Collector** in. The two roles most responsible for the campaign misreading itself — the writer of its own record, and the historian it never had — are now a **for** loop and a role that did not exist.



The loop after the 1 August rebuild. Three acts and a cross-round controller. Blue is an LLM call, green a deterministic check, orange code that is not an agent at all. **Thirteen roles in fourteen places** — the Biologist appears twice, static and probe before the batch (5) and passive after it but *before* the Reader (6), because a phenotype named from a specimen whose chemistry was extinct cannot be un-named. The roster is `ROLES.md` and `roles.py --check` compares it against the code in both directions.

ACT 1 — Propose. Before a second of compute is spent.

Grounder	agent	what did Okuda actually do? Reads the paper, with its own quotes checked	to build
Proposer	agent	which mechanism edits do we test next? Composes the batch under both mixture rules	runs
Peer-review	agent	is this batch <i>worth</i> the compute — falsifiable, not already settled, a mechanism rather than a restatement of the edit? Advises; cannot refuse	runs
Critic	check	can this be <i>run</i> ? Type-legal, preconditions met, not already evaluated, and every scheduled operator actually acted. Owens the in-distribution envelope : CFL, bounds, stability, admissible ranges	runs
Biologist	check	could this be a tissue? Against PREMISES.md, which it reads rather than restates. Categorical, never prose : <i>invalid</i> on a certain premise, <i>ambiguous</i> on a usual one. Static tier here (the spec alone) and probe (its own 40-frame sim); the passive tier runs in Act 2	runs

ACT 2 — Measure. The batch runs. The only expensive step.

Biologist	check	<i>was</i> this run a tissue? The same premises, now against what the run recorded. Runs after the batch and BEFORE the Reader — run it after the analysis and a phenotype has already been named from a specimen whose chemistry was extinct, which is what happened on r002c_00. Nothing can un-name a phenotype	runs
Metrologist	check	which metrics are admissible? Certifies instru-	runs

Dropped: Judge, Referee, Evolution, the duplicate check. The Judge existed to settle the Eye-check against the number, and demoting the Eye-check to observation dissolves the dispute; the Referee ranked by tournament what a certified metric already ranks. **Both were called zero times.** **Evolution** was asked “*what should change next?*” — and so was the Meta-review. Two agents answering one question is not a redundant call, it is a roster nobody can reason about. The surviving split is by **scope**: Meta-review over *this batch*, the Archivist over *the whole history*; local refinement of a winner inside the current branch was the weakest of the three jobs and is the one that went.

And the missing role was the historian. Nothing in the roster ever looked across rounds, which is why the search could drift down a line with nothing able to say so — and why the Proposer could record “*parent 2 is fully PROPOSED*”, counting a family as explored because it had been *proposed* rather than because anything had been *measured*. Only the history knows which. The **Archivist** is also what makes the abort path actionable: a round with no evidence routes back through it, so the loop can roll back to a better branch instead of re-proposing inside the envelope the Critic just refused.

Where the hypothesis lives, and who decides whether it held. This is the discipline the whole method rests on, and neither paper does it: in Co-Scientist and Robin a hypothesis is generated, reviewed and ranked, but nothing records what was *believed before the evidence arrived* — so a prediction cannot be told apart from a rationalisation written afterwards.

step	who	what
register	Proposer	every slot states a claim, an intent, a named metric and a prediction <i>containing a number</i> — <code>protr_peak >= 2.0</code> — posed under an id that cannot be overwritten, before anything is submitted
admit the metric	Metrologist	a prediction may only name a metric certified against known answers. A prediction on an uncertified metric is not a hypothesis, it is a wish
catch the uncheckable	Peer-review	the only role whose job is to spot a prediction that <i>cannot fail</i> before the compute is spent
score	code (predict.py)	checked against the measurement arithmetically. No agent decides whether a hypothesis was validated
use	Supervisor	the surprise rate — how often the prediction was wrong — drives the next batch’s mixture

Three outcomes, not two: *confirmed*, *refuted*, and **inconclusive** — a prediction the code cannot check. Inconclusive is not a soft refutation: it **drops out of the surprise denominator entirely**, because a prediction that could not fail teaches nothing and must not dilute the rate that steers the campaign. The scorer is code for the same reason everything else here is: a hypothesis scored by an agent is a hypothesis scored by the same kind of thing that wrote it, and the surprise rate is too central to sit inside a model’s discretion.

Which roles serve which track. The roster is machinery; it exists for two things at once, and a role that does not know which one it serves will drift.

	Track A — understand the mechanism	Track B — reproduce Okuda
the product	a map : which operator does what	the figure : tube, undulation, branching
measured by	lever-map coverage, and the surprise rate	the scoreboard: how many of the four morphologies
served by	Critic (what is legal), Reader (what happened), Archivist (where to search)	Grounder (what Okuda did), Biologist (is it a tissue at all)
fails by	confirming what we already believe	producing the picture by hand-tuning, which proves nothing about the method

The 70/30 in-distribution rule is where the two tracks are allocated, and that is its real purpose. `in_paper` slots serve Track B; `excursion` slots serve Track A. Running only Okuda’s settings teaches us to reproduce his figures rather than to understand the system — the objective is a *map*, *not a target* — and running only excursions produces understanding of a model nobody has a reason to believe.

The two mixture rules, each with an owner. Both are properties of a *batch*, so both are checked before compute is spent:

rule	composed by	checked by
70 confirmatory / 30 adversarial	Proposer	Peer-review
70 in-distribution / 30 out-of-distribution	Proposer	Critic, deterministically

In distribution means the parameter and solver-validity envelope — CFL, bounds, stability, numerical sanity — which is checkable without judgement, and is why it is the Critic’s. It does *not* mean the composition family already explored (the lever map’s), nor the regime where the metrics are certified (the Metrologist’s).

A round that produces no evidence is ABORTED, and does not advance to Act 3. It routes back to Act 1 carrying the Critic’s refusal reasons. It consumes budget and is recorded in full, but **does not increment the round counter and enters no coverage denominator** — counting it as a normal round is exactly how a log came to tell the Proposer **coverage 0%**, from which it correctly concluded that its own ledger was broken. **Two consecutive aborts stop the campaign and ask:** a second abort means the refusal reasons were not actionable, which is a fact about us rather than about the search, and without that bound “route back to Act 1” is a loop that burns a week.

What the live run actually spent, which is how the roster was judged. Sixteen calls, 23.5 minutes: **Proposer 14.8 min** (63% of all agent time), Peer-review 2.8, Meta-review 2.4, Readers 1.7, and one call each to the Eye-check, Interpreter and Evolution. **The Judge and the Referee were never called at all.** That is the measurement the redesign rests on — not an argument that they were redundant, but a ledger showing they had nothing to do. Timing was wired on 1 August; until then nine call sites bypassed the stopwatch and a round once reported **calls: 0** for a round that made about twenty-five. Six real reader calls came in at **21 seconds** each against a four-minute allowance and **7.2 turns** against a cap of 60, so neither the timeout nor the turn cap has ever constrained anything.

7. What stands between us and the Okuda figure

Most of the gap is mundane and it is about starting conditions. We were running 150 cells where Okuda starts at 200, 2000 and 4000, and because the mesh buffer is sized as a multiple of the starting count, beginning at 150 capped every run at 1778 cells — so his largest case was not reachable at all. Two of his settings also sat outside the box our search was allowed to use: how sharply a cell switches on its growth (he uses 10, we allowed up to 8) and how much further the inhibitor travels than the activator (he uses 10, we allowed 2), and the second is why we could not produce his handful of well-separated spots. A third number, his pattern length-scale, is not even the same quantity as ours, so the decision taken with Cedric is to calibrate against what he *reports seeing* — the spot count on a ball of a given size — rather than copy a value across two differently-scaled models. All of that is four numbers and a buffer, and it is closed in Phase 2. The one genuine gap is that the coupling only runs one way: chemistry deforms the tissue, but the tissue's shape never feeds back into the chemistry, because our diffusion averages neighbours equally instead of weighting by shared wall area and cell volume. A plain tube probably does not need that arrow — a tube needs a spot that stays put, and a decal does that perfectly well. Branching almost certainly does need it, because in Okuda's account a tube forks when the tip stretches, dilutes its chemical unevenly, and the domain splits. So without it we would not get branching, and — worse — we would not know whether that was a real result or our missing arrow.

8. The phases

Eight stages. **I stop at the end of each one and wait for you.** That is the partition.

Phase 0 — Pre-flight: fix the instruments

done (28 of 28)

Goal. Make every measurement trustworthy. Nothing runs unattended until this lands. Not one of the twenty-eight items is science; each one makes a question *askable*.

The last two, closed. *Comparing a batch at a common frame* — the per-run evidence horizon was right and did not make a BATCH comparable. If one run tears at frame 300 and another holds to 700, comparing their “final” states puts a torn mesh beside a healthy one and reports the difference as biology: both operands valid, the comparison wrong, which is the shape of every error that has cost this campaign a batch. A batch is now read at one frame, the earliest horizon in it, and the tool reports what that costs — how much of each run is discarded, and a warning when one run's early death is being imposed on everyone. *The explicit vocabulary* — `elongation`, `elongation_peak`, `elongation_at_end`, `elongation_unforced`. Writers emit both names, readers resolve either, the archive is never rewritten: cutting 92 references over in one step leaves a window where a key is written by half the code and read by the other half, and since an absent metric reads as “not measured”, the bug would surface as a round that quietly learns nothing.

The repairs worth knowing about

what	what was found
survival test	It relaxed a fresh sphere and returned 1.014 for every run. Now real: 1.415 → 1.325 , so 94% of the elongation survived the drivers going off — the first honest survival number the campaign has produced.
the camera	Growth was drawn as <i>shrinkage</i> : apparent size fell to 0.52× on a run that grew by half. A second viewpoint justified itself immediately — in one run the tube is invisible from the side and obvious from the top .
the tube bar	No capsule of any aspect ratio can reach the old bar of 2.0 (ceiling 1.90). The criterion was not asking for a tube; it was asking for the spike artefact.
damage	The blended damage score was counting <i>cell division</i> : sliver correlates with tip count at +0.94, while genuinely broken cells stayed at 0 for 300 frames .
the 1778 ceiling	Arithmetic, not physics: $(3552+4)/2 = 1778$, and all 32 runs of the overnight study stopped there.
the missing arrow	The emitter never passed the implementation through — the ablation would have reported “no effect” <i>without ever running the new code</i> .
the curve plot	Drawn every run since the beginning, referenced nowhere in the codebase .
timing	The round reported calls : 0 for a round that made about 25.

Phase 1 — Teach the loop what a tissue is: the premise gate

done

Why this became its own phase. Ten defects were found in one day, every one by Cedric looking at a picture, and all of the same kind: an operator or an instrument not doing what its name says. The loop checked that runs finished, that the mesh was valid, that the numbers were finite. It never once checked that the thing it had simulated was a tissue.

Eleven basics, each one something a computer can run and fail, in three tiers by cost: read the recipe (instant, refuses a hopeless run before a GPU is touched); read the run (free, goes loudly into the record); and run a small experiment of its own — switch everything off and let the ball sit for forty frames; it must not change size.

A broken premise must be declared, not silenced. Switching off growth to ask what a protrusion is made of genuinely breaks premise 1, and that is science. So a recipe may waive a premise *in writing, with a reason*; an undeclared violation is a hard stop. That is the only thing separating “we tested an idea” from “the settings were wrong”.

Every check was proved to fail before being trusted. The one to look at: run against the code as it stood that morning, a resting ball collapses **5.00** → **0.69** in forty frames. That single check, which costs about a minute, would have caught the worst defect of the campaign on day one.

Phase 2 — Make Okuda’s settings reachable

done (6 of 6)

The box used to describe the work without listing it, so the count had no visible denominator. All six, with what each one turned out to be:

. item	state	what it turned out to be
0. The ball could not get big enough	done	<i>Not on the list.</i> A spring held the shell at the size it was first built at while the cells inside grew sixteen-fold, so the sheet folded through itself. The spring's target now follows what the cells are asking for: the same run grows smoothly to 5 240 cells and stays clean.
1. Widen the out-of-range settings	done	Done, and the new bounds are <i>derived</i> rather than chosen: growth sharpness to 49.5 (Okuda needs 10), inhibitor spread to 12.5 (he needs 10), reaction speed unbounded.
2. Replace the mislabelled length-scale	done	The quantity we called a pattern size is a solver rate. Resolved in Phase 4 by measuring the pattern instead of setting it.
3. The shape-to-chemistry feedback	done	One contract, four implementations — curvature, tension, apical area, pressure — because <i>which</i> feature the chemistry listens to is the hypothesis, not a dial. Force and size were refused with reasons.
4. A fixture asserting his published constants	done	His Table 1 is now written down in the Grounder, quoted, with the sentence each value comes from — the Hill coefficient, the two diffusivities, the cell cycle, and the two regime <i>inequalities</i> that actually define his setting. Asserting a run against it is Phase 3, because it needs the Grounder wired.
5. His cell counts, with the buffer sized for the <i>destination</i>	done	The item that voided the weekend batch , and it is measured rather than argued — see below.

Item 5, tested rather than argued. A closed cell sheet is trivalent, so Euler fixes the vertex count at twice the cell count: a reservoir of 3552 vertices caps the tissue at $(3552+4)/2 = 1778$ cells exactly, whatever the biology wants. The reservoir is now computed from the count a run is *aiming at*. The same recipe, run twice, with nothing changed but the seed count and the reservoir:

	seeds	vertex buffer	cells at end	verdict
before	150	3 552	1778	saturated — not evidence
after	200	10 404	4151	clean — valid evidence

The old configuration stops dead on the rail; the new one grows past Okuda's largest case. Elongation is 1.061 against 1.073, so the physics is unchanged — the run simply gets to happen. Sizing the buffer from the seed instead of the destination is what produced 1778 twice in one week.

One error the check caught on me. My first version sized the reservoir from the seed count, which gave the 200-cell case a ceiling of 264. Okuda is explicit that his largest tissues were “picked up on the growth process” — a destination, not a start — so the two are now recorded as separate numbers.

Certification caught two of my own errors on item 3 — a curvature normalisation that only looked right because the test held cell count fixed, and a test of mine that demanded the wrong answer.

The phase closes on a green self-test, which it did not have an hour ago. The composition space checks itself in 61 ways, and eight were failing — silently, since yesterday. Seven were the growth-ceiling fix and the chemistry-clock fix diverging from the archived recipes: the space is right and the archive is wrong, so both are now *declared* deliberate

rather than back-fitted into the archive, because rewriting a reference to make a test pass is how it stops being one. The eighth was real — no reference recipe used the new shape-to-chemistry operator, so the recipe named for Okuda’s route was missing the half that makes it his. It carries it now. And the chemistry clock, previously exempt and unchecked, gained a test of its own: both terms must scale by exactly $1/dt$ *and* their ratio must be unchanged, since that ratio is what sets the Turing wavelength. **61 of 61.**

Phase 3 — Switch on the agents that never run

done (3 of 3)

Ablation is now compulsory — *done*, and it is the part that matters. A claim that one thing *causes* another needs two experiments and we were enforcing neither: adding a mechanism and seeing the shape appear shows it is *sufficient*; taking it away and seeing the shape die shows it is *necessary*. Either alone is an opinion.

Every claim must now declare its kind, and the checker throws out a batch that asks for more than it tests — before any simulation starts. Sufficient earns “is sufficient for”; necessary earns “is required for”; both earn “causes”; neither earns “is associated with”. **The word can no longer outrun the experiment.**

Still to do — and one of the three is now the priority.

(a) Wire the Grounder — *done*. It is the only agent that reads the paper and it had no call site anywhere; every entry point was exercised only by its own smoke test. Batch construction now asks it, and it answers with Okuda’s own numbers: 2,000 cells, carrying the sentence they came from. It arrives as a *parameter*, so composition identity is unchanged — setting the starting size is a retune, not a new mechanism, and the search cannot mistake one for the other.

Two rules keep it from becoming a straitjacket. The Grounder *advises*: the faithful share of a batch inherits his numbers, the exploratory share is free to leave them — a search that may only ever stand where the paper stands cannot discover that the paper is not the only place to stand. What is *not* optional is the reservoir, and that lives in the translator where it applies to both shares alike.

And it was wrong the first time, in the most dangerous way available. My first version started tubulation at 200 cells, quoting the paper. The sentence was real; the mapping was not. “About 200 cells” belongs to two *control* experiments Okuda runs first — one on arrested tissue that cannot grow at all, one with growth decoupled from the morphogen. His three morphology figures are a single coupling experiment at **2,000** cells, and only χ and γ separate them. Cedric caught it. A quotation is not a citation: the quote made a tenfold error look verified, which is worse than having offered no quote at all. There is now no case to infer — a slot either declares which of Okuda’s experiments it is, or it gets the coupling one.

Still open here: asserting a run against his published constants. The values are written down; nothing yet checks a run against them.

(b) Evolution and the Referee — *done*. `rank_bt1` had been written and self-tested since the loop was built and had never ranked a real batch; the round sorted on whichever scalar the instrument gate happened to admit. A tournament decides now. Verified on a synthetic batch: a vetoed run holding the highest score drops from first to last. Evolution runs too, refining the winner rather than the loop only ever *picking* from what the enumerator offered.

(c) The per-run calls run together — *done, and measured*. Runs are read concurrently across a batch, and the machinery for reading one run N ways in parallel remains, because no reader’s prompt depends on another’s answer.

N is now 1, and the reason it was three did not survive being examined. Three readings were justified by Robin’s eight, and Robin’s Finch *writes the analysis code* — it chooses the flow-cytometry gating and the RNA-seq filters, so its trajectories produce gen-

uinely different *numbers*. Ours writes none: by the time a Reader is called, `diag.json`, the curve shapes and the strip were computed by instruments the Metrologist certified, and every reader would see identical numbers. What varies is the *label* — phenotype, forced-or-grown, the concern raised — so N readings would measure **phenotype ambiguity**, a real quantity but a much smaller prize than the one the argument was imported for. What we give up at one, stated plainly: nothing records when a label was a close call. The count is a single number and raising it is one line.

The eye-check still runs *after*, because it is handed the reading and is not independent — parallelising a dependency is not an optimisation, it is a different experiment.

Phase 4 — Measurements that can tell his regimes apart

[done](#)

The classifier — *done*. Sphere, undulation, tube, branched, plus **invalid** (the surface passes through itself, so no shape reading means anything) and **unclear**, returned deliberately near a boundary. A classifier that always answers can never be wrong, and one that can never be wrong teaches nothing. “Invalid” beats every morphology — this campaign has already once reported a crumpled shell as a result. It records the *path* a run takes, not just where it ends.

It has since become the arbiter (Phase 4d), replacing the labels a person wrote from the movies — a stronger ground truth, not merely a cheaper one, because it is certified against shapes whose answer is known *by construction*. Until 1 August nothing called it: built, certified, never asked. Every run now carries its shape and its path beside its numbers.

The calibration, and the surprise. The plan was to tune until we saw Okuda’s handful of spots on a 2000-cell ball, then freeze. Counting spots over time — for the first time — showed something else:

step	0	50	100	200	400	3000
spots	104	62	19	7	1	1
coverage	0.06	0.14	0.37	0.49	0.52	0.53

The pattern never settles. It keeps merging until the ball is one connected blotch covering half the surface. So “about five spots” was a passing moment around step 200, not a property of the settings — and calibrating to it by choosing when to stop would have produced a number that looked right and meant nothing. The chemistry this campaign has always used makes *mazes*, not spots. Nobody noticed because nothing counted spots, and the movie’s colour scale was rescaled every frame, so a merging pattern and a stable one looked identical in both the numbers and the pictures.

The calibration became a search for where stable spots actually live, tested by counting at two well-separated times. Twenty settings; the winner gives three spots that stay three spots across three independent runs, and is frozen as the default.

Phase 4b — Can the chemistry be simulated at all?

[done](#)

Why this became a phase, mid-flight. The pre-launch stopped on a run whose chemistry turned to infinities within a hundred frames. Chasing that found the rule which exists to prevent exactly this, and it had been **fifty times too generous** for as long as the campaign has existed.

What the rule is for. Two chemicals sit on the cell sheet: one that builds up and makes more of itself, and one that spreads further and suppresses it. The simulation moves them a little at a time. If a single step moves too much at once, the arithmetic runs away and every number becomes infinite — and everything measured afterwards describes the solver rather

than the tissue. There is a standard condition for how much movement per step is too much. It depends on two settings: a *spreading multiplier* that scales both chemicals together, and how far the suppressing chemical travels.

What went wrong. The condition was written down as a *comment*, worked out once, for the settings of the day. Then the defaults changed, and Phase 2 deliberately widened the range so Okuda’s own values would be reachable at all — and the arithmetic in the comment quietly stopped describing the code. A comment cannot be re-derived. It can only be re-read, and nobody had cause to re-read it. On top of that it counted one factor twice, reporting a fiftieth of the true figure.

What it cost. `okuda_route` — the recipe named for the target — was four times past the safe limit and passed every check in the system. Its chemistry died within a hundred frames, and the Biologist refused every run that used it. That refusal is the only reason this was found: no metric would have shown it, because there were no numbers left to look at.

Then it was tested against the engine, and the corrected rule was still wrong. Twenty runs on the cluster, three hundred frames each, across the predicted boundary. An argument about where a limit lies is worth nothing until the boundary it predicts is the one the engine has:

spreading multiplier	suppressor range	what actually happened	
0.01	10	healthy pattern	Okuda’s own value works the working band
1.3	0.16–0.42	healthy pattern	
3	0.16	healthy pattern	
0.05	10	infinities	predicted safe
1.3	0.5	infinities	
4	0.16	fine, but dead	no pattern at all
6	0.16	fine, but dead	

One number cannot express the limit, because there are two ways to fail and it knows about one. Running away is the one it forbids. The other is that the chemicals spread so readily that the pattern smears out and dies — the simulation stays perfectly well behaved and measures nothing. **Well behaved is not the same as alive**, and no stability condition sees the difference.

So the rule is now two conditions, and both were measured rather than derived. The step size must be under the limit, *and* the spreading multiplier must stay under three. The second has no formula behind it — it came from watching fourteen settings run. It exists because the multiplier scales both chemicals at once, so turning it up flattens the self-reinforcing chemical’s own peak as fast as it spreads the suppressor, and past about three the peak cannot outrun its own spreading.

Okuda’s own setting for how far the suppressor travels *is* reachable — it sits in the first row of the table above, at a low spreading multiplier.

The limit sits below the first failure we saw. The measurements do not fall either side of a clean line — two settings above it ran fine, one below it did not — so the rule refuses two combinations known to be good.

That is the safe direction, and it is worth being explicit about which one is which. A *permissive* rule admits a setting that then fails: the run happens, produces numbers, and those numbers are about the solver. That is how thirty-two runs and then twenty-seven more became findings before anyone noticed. A *strict* rule refuses a setting that would have been fine, and costs one proposal. The old rule was permissive by a factor of fifty; this one errs strict until the term that explains the odd case is found.

Checked against all eight settings that were run: every bad one refused, no bad one admitted. **Still to do**

- **Find the missing term.** Two settings cannot order the failures on their own, and the rule stays conservative until something explains the case that breaks the pattern.
- **The reaction has no limit at all**, on purpose — a candidate was proposed, measured and refuted — so a recipe can pass this check and still die through the reaction instead.
- **Longer runs may move the boundary.** These were judged at three hundred frames. A run that survives three hundred and dies at nine hundred is still a run the gate let through.
- **One trusted recipe sits on the wrong side.** `round_44_base` ran healthily by hand — nine hundred frames, three thousand cells, a strong pattern — and is four times past the limit when rebuilt as a composition, because the clock repair speeds its chemistry up fiftyfold. Both are true, and they are not the same experiment: the archived run had a fiftieth of the reaction time the repair gives, which is the very defect the repair was written for. Keeping it as a reference keeps a pre-repair run among the trusted ones. The self-test sits at 54 of 56 on exactly this, and the decision is Cedric's.

Whose job this is. The **Critic**. It is deterministic, it runs before any compute, and it already refuses a recipe that is illegal, unrunnable or already tried — “cannot be simulated” is the same kind of statement. Not the Metrologist, who certifies that an *instrument* measures what it claims, and not the Biologist, who asks whether the *specimen* is a tissue. This one asks whether the arithmetic can be carried out at all, which is a property of the proposal, and the proposal is what the Critic reads.

Phase 4c — Multi-spot initialisation, and replication

done

Why. Cedric: “*did you forget to run multiple tubes from now on, not a single one*”. He is right, and the note already said so — one protrusion per run is a single sample, and you cannot tell “this mechanism makes tubes” from “this run made a tube”. Several spots give within-run replication for free, and it is what Okuda shows. Every control I built during the pre-launch used one spot or none.

Measured, six runs at the stable point, 300 frames:

seed fraction	act_max	activated fraction	
0.02	0.423	0.40	
0.06	0.501	0.37	three seeds, identical to 3 d.p.
0.15	0.598	0.12	
0.30	0.000	0.10	the pattern is extinguished (P4)

Two findings, one of them a problem. Seeding more widely does not give more pattern — it gives *less*, and past 0.15 it gives none at all: a seed covering a third of the shell leaves no room for the inhibitor to separate domains, so the whole field decays together. The useful band is narrow and sits below 0.15.

And the three seeds were bit-identical — 0.501, 0.501, 0.501. `general.seed` was written into every spec and read by *nothing*: both stochastic operators take a `seed` (the mesh jitter, and which cells are nucleated) and the translator passed neither. “Three independent runs” were one run repeated, so every spread this campaign has quoted across seeds described the floating-point reproducibility of a single trajectory.

Fixed, and the fix follows the working specs rather than my first idea. I offset the two operators' seeds, reasoning that jitter and nucleation are independent draws. True, and beside the point: `archive_rounded.py`, `archive_vh_rd_coral.py` and `run_tyssue_fig5.py`

all pass *one shared* SEED to both, and a composition that reproduces an archived run has to draw the numbers that run drew. Departing from the convention would have made every reproduction check compare two different experiments. A spec that declares a seed no operator consumes is now refused outright.

Replication, measured for the first time — three seeds, 300 frames, same composition:

	act_max	activated fraction	elongation
before	0.501, 0.501, 0.501	0.374×3	1.006×3
after	0.501, 0.447, 0.483	0.407, 0.379, 0.374	1.006, 1.006, 1.005
	sd 0.022	sd 0.015	sd 0.0005

That last row is what a difference between two compositions now has to beat to mean anything. It did not exist yesterday, and every claim made without it was comparing single samples.

Phase 4d — The picture stops voting

done

Cedric's call, and it removes what had been blocking the loop. The metric bank was certified by making numbers reproduce labels written from the *movies*, and a vision model's reading of a movie could drop a run out of the ranking entirely. Movies stay — the model still watches every run, and a disagreement between the picture and the numbers is the only signal in the loop that comes from looking at shape rather than at numbers. What they no longer do is *decide*.

Two reasons beyond the call. The veto was circular: the eye-check was handed the reader's label — derived from the same caption it then read — so it could catch a summariser drifting from a caption and could not catch numbers disagreeing with reality, which is what it was credited with. And the camera was broken for the whole period the veto was trusted: one fixed viewpoint, and a zoom re-fitted to the tissue every frame, so growth was drawn as shrinkage and a tube could sit behind the body. A gate reading a broken instrument is worse than no gate.

What arbitrates instead. The morphology classifier, which answers *which of Okuda's shapes is this* from geometry and is certified against shapes whose answer is known by construction — a built sphere is a sphere, nine built bumps are an undulation, an extruded tube is a tube, a forked one is branched, and a self-intersecting shell is refused a label at all. Nobody's opinion enters. Alongside it, the Biologist: a run that fails a premise is not a tissue and cannot calibrate an instrument.

What this costs, honestly. The eye-check vetoing the top two runs of round 2 was reported as the system's best moment. It becomes a *recorded disagreement* instead — which is the truthful version, since the camera it was reading was showing growth as shrinkage at the time. The disagreement is still written down, and it is still worth reading.

Phase 5 — The first live run: the agents do not talk to each other

IN

PROGRESS

It ran, and that is the point. Two rounds, eight simulations, 23 agent calls, 29 minutes of agent time. **One usable run. Map coverage 0%.** Nothing was learned about the biology — and the reason turns out to be worth more than the biology would have been.

What the machine got right, unprompted. The Critic refused every bad run and named the reason. The straggler killer cut two runs at 46 minutes against an 11-minute batch median. The Supervisor diagnosed its own sterility — *"surprise 0.00: the batch is*

confirming what we already believe, near-zero information. Push adversarial” — and a round with nothing admissible said so instead of scoring what it had. Every one of those is a correct judgement. **Not one of them reached another agent.**

The diagnosis: written by somebody, read by nobody

The Proposer’s own reasoning for round 4, verbatim, is the whole case:

“Header shows round 3, 0 runs, coverage 0% ... another **counter-reset artifact**; real record = memory.md.”
 “Those two batches proposed ALL FOUR valid single-op edits on parent 2, so parent 2 is **fully PROPOSED** and ... cannot be built on further.”

It was handed a summary built only from the lever map, and the lever map is filled *after* the Critic — so a refused run contributes nothing to it. Eight simulations had died of diverged chemistry and saturated buffers, and the Proposer was told “0 runs, coverage 0%”. It then drew the only sane conclusion available from an insane input: that the *ledger* was broken, and its own prose memory was the real record. And it counted a family as explored because it had **proposed** the edits, not because it had **measured** anything — so it moved on to a fresh parent believing the previous one was mapped, when nothing had been learned there at all.

An audit of every hand-off found the same shape seven more times:

who produces it	what	who receives it
Critic	every refusal, with its reason	nobody
Supervisor	“push adversarial”, the mix steer	the terminal, only
Interpreter	causal_descriptions.md	nobody
Evolution	evolution.jsonl, its refinement	nobody
Grounder	Okuda’s starting conditions	the config, not the Proposer
scoreboard	scoreboard.md, the headline product	never written by a round
lever map	lever_map.md	nobody

And the instruction the Proposer works from names three admitted metrics, of which the **ta_*** family is **absent from half the runs on disk** — so it writes predictions that cannot be scored, which resolve *inconclusive* and drop out of the surprise rate entirely. The loop is measuring its own information rate with a ruler that discards the measurements.

The reason, stated once

The campaign was built as a set of roles, each with a job, each doing it. What was never built is the *wiring*. Every agent writes; almost nothing reads. The result is not a loop but a fan: work flows outward from the Proposer and never returns, so the system cannot be corrected by its own findings. **A loop that is told only about its successes cannot avoid repeating its failures** — and this one had no successes to be told about.

That also explains the day’s other defects, which looked unrelated and are not: a refusal printed among hundreds of lines, an evidence horizon silently absent, a buffer flag raised after the compute was spent, a state file grown past reading, a black movie. Each is a correct output with no recipient.

Drawing it is what made the shape obvious. Every agent’s output was plotted against whoever read it, and the picture was a fan rather than a loop: work flowed outward from the Proposer and almost nothing came back. Fourteen roles, and the only thing they shared was

the direction of travel. That picture is what turned the repair from “wire these hand-offs” into “why are there so many hand-offs to wire” — which is §6, and Phase 6.

Fixed

Refusals are now evidence. The Proposer receives what ran, what was refused and the reason code, with the distinction stated plainly: an empty map beside a non-zero attempt count is not a reset counter — it means the compositions proposed so far *cannot be simulated*, a parent is explored when its edits produced evidence and never when they were merely proposed, and the refusal codes say what to change.

The black movie. The colour scale is computed once for the whole run — correctly, since a per-frame rescale makes a decaying pattern and a stable one look identical — but over samples including non-finite values, and one of those makes both bounds non-finite. **Frame 0 was perfectly good and was drawn black because frame 800 diverged.** The artefact reported “nothing happened” for a run whose first half was fine. Now: finite values only, cells that left the model drawn grey rather than white, and the count said out loud.

The Biologist now speaks to somebody. It had been running on every run since it was written, and its verdict went to a terminal and to a field of `diag.json` that no prompt has ever mentioned. On round 2 it broke five premises on the very run whose movie came out black — *the activator had decayed to NaN, the chemistry was extinct* — and a Reader read that run’s numbers and named a phenotype. Two hand-offs now exist: the per-run verdict goes to the **Reader** that is about to read it, in the document’s own words and with a field it must fill in saying whether the specimen was sound; and the round-level tally goes to the **Meta-review**, whose whole job is the recurring pattern. Across round 2 that tally reads “*P4 broken in 2 of 3 runs*” — an edit family that cannot hold chemistry, which is a thing to stop proposing rather than a thing to keep measuring.

And the premises are read from PREMISES.md, not retyped beside it. Two documents saying the same thing in different words will disagree, and the one that drifts is always the prose — it is the one nothing executes. The titles, grades and stated checks now come from the document, and the drift is reported in both directions. It found two immediately: premise 10 (neighbour exchange is rare) is written down and **nothing checks it**, and P12 is checked on every run and **appears nowhere in the document**.

The two written records have a shape, checked rather than suggested: an append-only log with per-round fields, beside a state document of seven named sections. `memory.md` had reached 1904 words across six appended blocks in a file whose purpose is to hold state.

What the diagnosis led to

Wiring the hand-offs one at a time was the wrong repair, and stopping to ask *why there were so many to wire* was the right one. The answer is in §6: sixteen roles copied from a system that runs no experiments. **The roster was rebuilt — thirteen roles, three acts, Judge and Referee dropped, the Collector made code** — and the design is written down in `ROLES.md` so that this decision cannot evaporate the way the Biologist’s verdicts did.

Still to wire

- **The Collector** — the round record, built from disk. Everything below lands on it.
- The Supervisor’s steer into the Proposer’s prompt, not the terminal.
- The Grounder into the Proposer’s prompt, rather than into the config.
- The Critic’s deterministic 70/30 in-distribution gate.

- The abort path: no evidence → back to Act 1, two in a row → stop.
- The Interpreter’s descriptions and Evolution’s refinement into the next round’s evidence.
- The admitted-metric list rebuilt from what runs actually record.
- The reservoir target from the growth a composition asks for — three of eight runs outgrew an estimate the gate then correctly refused, one step too late.
- The scoreboard written by the round that changes it.

Phase 6 — The rebuilt loop, and its first launch

DONE

Goal. Launch the three-act loop of §6 and let it run. Phase 5 diagnosed why the old one could not learn; this is the one that is allowed to try.

What was built

- **The Collector** (`collector.py`) — code, not an agent. It assembles the round record from the files on disk and **writes** `analysis.md`, which the Proposer used to write about itself. Every field is measured or quoted; it composes no prose. It also reports its own *holes* — `[collector] HOLE: r003c_01: the Biologist never ran on it` — because the point of building a record from disk is that a missing input becomes visible rather than silent.
- **The Archivist** (`archivist.py`) — the role the roster never had. It reads the whole history, answers *continue* / *roll back* / *stop*, and runs on every abort. All the arithmetic is done in code and handed over as a table; only the choice is put to a model.
- **The abort path.** A round with no admissible evidence no longer advances to Act 3: it is recorded in full, **does not increment the round counter**, routes back through the Archivist, and **two in a row stop the campaign** with exit code 3, which the driver treats as terminal rather than retrying.
- **The three return paths** the Proposer never had: the Supervisor’s steer, the Critic’s refusals *with their reasons*, and the Grounder’s starting conditions from the paper — each a prompt section that disappears when empty rather than saying “none” in ten lines.
- **Judge, Referee and Evolution deleted** from the code, not merely unwired. **Reader** ×1, renamed for what it does. The **Biologist** now returns a categorical verdict — *invalid* on a **certain** premise, *ambiguous* on a **usual** one — decided by the grade in `PREMISES.md` rather than by judgement.
- **memory.md moved to the Meta-review**, on an enforced template. Neither record is the Proposer’s any more.

What the cold start found, and what it cost to misread it

The loop now begins by asking the Archivist where to start from what is on disk. The first thing it did was read `mechanics.npz`, which nothing had ever read: **across all 63 finished runs, `n_protruding` never once exceeded zero and `protr_peak` never exceeded 1.13**. The summary had been reporting “best sound run 1.11”, which reads like a result and is a sphere with noise on it. **This project has never produced a protrusion**, and the number saying so sat in every run’s `mechanics` file, unread, for the whole campaign.

Then it gave up — and that was the expensive mistake, and it was ours. It ranked every candidate on `protr_peak` and `n_protruding`, found zeros, and fell back to `okuda_route`. Those are **Track B** measures: they say whether we made Okuda’s picture.

Track A asks a different question, about the landscape of operators, and a composition that runs, patterns and holds a sound specimen is a point on that map whether or not anything protruded. Judging a Track A starting point by a Track B criterion, and then treating a Track B zero as the end of the road, is how the fallback recipe became the starting point three separate times. **Measured on the corrected ranking: 46 of 63 runs serve Track A.** The campaign had 46 usable starting points and was reporting that it had none. *Track A does not run out* — so **stop** is now stated as a Track B verdict only.

Three defects the batches themselves surfaced

Two operators where one was meant. `impl_structural` marked an operator whose implementation is a real mechanism choice *and* was read as permission to hold two of them, so the menu offered both `+shape_energy_3d:default` and `=shape_energy_3d:default`. The Proposer took the add and wrote the claim for the swap; the spec came out with two whole mechanics solvers driving the same vertices. Closed in the generator and, separately, in the Critic — a guard that lives only in the generator is a guard against one way of being wrong. **A stale proposal, silently adopted.** `proposal.json` was six seconds older than the process that read it. The harness’s read-before-write rule blocked the overwrite, the agent read the file and reported “*already holds a complete, valid Round-1 proposal ... no rewrite is needed*”, then verified it against every rule and ran it. The guard against clobbering had become a mechanism for inheriting stale work. The file is now deleted before the call, so there is nothing to inherit.

And the one that killed the batch. The activator went $0.01 \rightarrow 1.41 \times 10^6 \rightarrow \text{NaN}$ **while spatially uniform** — spread 3.4×10^{-5} against a mean of 1.4×10^6 . A diffusion breach makes a checkerboard; a uniform blow-up is an ODE exploding. The CFL was 0.208 against a limit of 0.45 and was never the problem. There was a bound on diffusion and **none on the reaction**. One probe at `dt=1.0` disproved the clock theory that looked obvious, and the discriminator is single-variable across three runs: `gierer_meinhardt` explodes, `gray_scott` patterns — and `okuda_route`, the recipe named for the target, carries the former.

The Diagnostician, and why it is a role

Cedric’s objection, and it is correct: “*we cannot launch, crash, and do this every time*”. Every one of those diagnoses was done by hand. The Biologist can say a specimen is invalid and the Metrologist that an instrument is uncertified; **neither can say why the apparatus failed**, so a human did the forensics after every crash — which is not autonomy, it is manual work moved later. The **Diagnostician** computes when each series went non-finite, whether the divergence was *spatial or uniform*, which runs share the signature, and **what differs between a failing run and the nearest passing one** — the step that isolated `chi 65 vs 1.3` with every other parameter equal. It returns **cause**, **evidence**, **guard_to_add** and **stop|continue**, runs *before* the Interpreter, and can halt the campaign.

The launch: reconnaissance first

The first round **sets no hypotheses**. The Proposer picks six specs already on disk from the Archivist’s ranked table and they are re-run *verbatim* — no edits, no predictions, intent **control**, resolving **inconclusive** so they leave the surprise denominator. There is nothing to be surprised by in a replay, and a prediction attached to one would be a prediction about our own past arithmetic. Every number we held was taken with at least one instrument since proved wrong; the product is a frontier of compositions we have *seen work*, measured with instruments we are entitled to believe.

It ran. Three rounds, then the same six specs again

	when	evidence	refused	specimens
round 1	22:34	3	3	all valid
round 2	22:55	2	4	all valid
round 3	23:16	3	3	all valid

Three *distinct* rounds — r001n, r002n, r003n all on disk together, the counter advancing — at ~21 minutes each, of which ~5.5 is agent time. Every act ran every round: Collector, Interpreter correctly declining a replay, Meta-review, Archivist. The refusals are P2_BUFFER_SATURATED on the stale-reservoir specs: the Critic working, not a failure.

The moment worth the day was round 2, the first time the Archivist had history to reason over: “*continue — sole branch is sound 3/3, best 1.01 sphere — a valid Track A operator point; no better branch*”. It read a sphere at `protr_peak` 1.01 as a legitimate point on the operator map rather than a failure. Ranking exactly that number *as* a failure is what made the cold start give up three times over.

And then Cedric asked why it kept running the same spec. It did: four rounds, the same six or seven runs with a swap or two. Three things made that inevitable and none of them is a bug — the Archivist’s ranked table is **static**, since re-measuring a run does not change its pattern quality; nothing told the Proposer what it had *already replayed*; and recon poses no predictions, so the surprise rate stays null and the map cannot grow. The Archivist had said so itself in round 2, in one phrase: “*sole branch*”.

Reconnaissance is a one-shot and I had it rolling to forty. One round establishes the frontier; every round after it must be **composition**, which is the mode that proposes an edit, predicts a number and can be *wrong* about it. `campaign_loop.py` is now phased — `--recon-rounds` defaults to 1 — and the two shell drivers written beside it are deleted. Bash cost two errors in an evening, one of them editing a script *while bash was executing it*, which re-reads by byte offset and corrupted the running instance.

What is running now, and what it is a test of

Twenty rounds: one reconnaissance to rebuild the frontier, then composition. **The first composition round is the first time anything in this project poses a falsifiable prediction and can be wrong about it** — and the first use of the parents that reconnaissance produced, now that `composition.json` ships beside every spec.

What to watch: whether the surprise rate is ever non-null (a campaign that only confirms has bought coverage and no knowledge), the Collector’s holes, and whether any run reports a protruding cell — which would be the first in the project’s history, across 63 prior runs and these.

Closing Phase 6

What was run

2 August 2026. One clean start at 13:02, one resume at 17:32 after an intervention, a clean stop at 20:35 — driver halted at a round boundary so the last round could finish writing its record rather than being killed mid-write.

Nine rounds recorded: 1–5 and 7–10. Round 6 was a reconnaissance round deliberately killed: on `-resume` the loop recomputed its plan from scratch and opened a campaign already five rounds deep in *recon* mode, which would have replayed old specs instead of breeding from the frontier.

79 hypotheses posed, 58 admitted as evidence, 21 refused. 66 simulations reached the cluster. Rounds cost 49–55 minutes of wall clock and 5–33 minutes of agent time.

Ten defects found and fixed while it ran, across 26 commits. The two that mattered most were invisible from the terminal:

- **The reading loop recorded one run per round, and that run was a chimera.** An indentation de-indent put `rows.append` *outside* the `for` loop, so scoring, prediction and recording ran once on leaked loop variables: the name of the last posed slot, the hypothesis of the last slot, the summary of the last run that survived the Critic, and the reader verdict of a third. Five clean runs were read successfully and none of them reached the record.
- **The ranking never saw a broken premise.** The sort key asked `summary` for `premises_broken`, which lives at the top level of `diag.json` — so it evaluated false for every run ever ranked. The invalid specimens have the highest `protr_peak`, so they won the ranking, became the frontier, and were bred from for five rounds. The demotion existed in the code and had never once fired.

The remainder: a straggler killer that could never spare a run (three separate causes, in succession — it read a file written only at the end; it was called with a set where it expected a mapping; it compared frame numbers across polls shorter than one frame); reservoir decisions that reached no agent; an exit code shared between “the search is stuck” and “the apparatus is broken”; a heartbeat reporting the array size instead of the cell count.

Results, and what we conclude

The machinery works. The surprise rate is *not* null: of 53 checkable predictions, **30 were refuted and 23 confirmed**. The loop poses claims it can be wrong about and is wrong about most of them, which is the only condition under which a round buys knowledge. Records now close arithmetically — posed equals evidence plus refused, every round.

Tubes appeared, on sound specimens. 42 of 66 runs are valid specimens; the best is r007c_02 at protr_peak 1.295 with two tubes, against a prior best of 1.169 with none. ad4767 returned 1.190 with one tube in three separate rounds — reproducible, not a one-off. The agents read these correctly and unprompted: “*a robust bud extending into an elongated fission neck*”, and, of a control, “*‘unchanged control’ is FALSE — parent buds under uniform ungated growth*”.

But the ceiling was set in round 3 and never moved. Seven subsequent rounds re-found 1.295 and never exceeded it. The reason is the finding of the phase:

Two of 66 runs contained a reaction-diffusion operator. Not one edit, in 42, ever tuned or added one.

The valid frontier carries cell_rd_seed and morphogen_growth_3d but neither cell_diffuse nor cell_react. Its activator is frozen and binary — across 900 frames the 95th percentile and the minimum sit at exactly 0.020 while a small patch sits at 1.0, with nothing in between and no gradient at any time. The patch does not grow with the bud; it *collapses*, from 75 cells to 8, while protr_peak rises to 1.170. What the campaign optimised was a mechanical instability decorated with a static red dot. The Turing half of Okuda was never running.

Three runs, put side by side, say what actually governs the bud — and no instrument in the loop can see it. A fine labyrinthine pattern gives a perfect sphere (1.003); a few coarse spots give a multi-lobed body (1.168); a single localised patch gives one clean bud with a neck (1.173). Localisation controls morphogenesis, because a fine pattern spreads growth evenly and even growth keeps a sphere spherical. There is no measure of wavelength, domain count or contrast, so the governing variable is not merely unmeasured — it is **unrepresentable**, and the run with the finest pattern in the campaign is recorded as morphology=sphere: a null.

Conclusion. The apparatus is sound and the epistemics are not. The loop reasons correctly from what it can measure, and its errors are all of one family: it converts absence of evidence into evidence of absence. It closed the reaction-diffusion family from a set in which every divergent run shared an unvaried confounder; it wrote *fundamental* of a ceiling it had never pushed on; it kept a necessity claim after its own round refuted it; and it recorded *sphere* where it should have recorded *not measured*. In its memory, four of seven positive claims carry a falsifier and two of nine prohibitions carry any escape at all — it knows how to make a positive refutable and does not do it for a negative. Untested territory has nowhere to live, so it collapses into *impossible*: this is how *we never tried it* became *the only remaining route is a different geometry*. That is not a defect to patch. It is the subject of Phase 7.

The main conclusion: the logic did not emerge

The reasoning discipline was never in the loop. It did not appear on its own, and no number of capable agents produced it.

This is the finding of Phase 6, and it is larger than any result about buds. The agents write fluently, read images honestly, diagnose an integrator instability unaided and correctly, and catch their own control misbehaving. What they never once do is ask whether a conclusion has been *earned*.

The record measures it exactly. Across `memory.md` and `analysis.md` — some forty-four kilobytes of agent-written reasoning over nine rounds — there is:

not one occurrence of *only tested once, may not generalise, confounder, insufficient evidence, or cannot conclude*;
against **fifteen** assertions of closure — six INERT, three NECESSARY, two FUNDAMENTAL, one EXHAUSTED, three prohibitions.

Fifteen universal and necessity claims, zero qualifications. Not a single hedge in the entire campaign. The loop has a role that asks *why did the apparatus fail?* — the Diagnostician, which works, and which correctly separated an explicit-Euler reaction instability from a diffusion one without being told. It has no role, and no rule, that asks *is this inference sound?* So instrument faults were caught and reasoning faults were not; every one of the eight was found by a person reading the record afterwards. The tempting reading is that the models were careless. They were not. Where the format demanded rigour they supplied it — four of seven positive claims carry an explicit falsifier, because the template has a **Falsifiable** `by:` field for positives. Where nothing demanded it, none appeared, because the template has no field for the conditions of a negative and no bucket for the untested. **The agents were exactly as rigorous as the structure they were given, and not one degree more.**

That is the lesson to carry: logic is not an emergent property of putting capable models in a loop. It is a *structure* — modalities, quantifiers, conditions attached to negatives, a place for what is merely untried — and if it is not written down and mechanically checked, the loop will run for nine rounds, produce a genuine refutation rate of 57%, and still conclude that a region it never entered is impossible.

Hence `LOGIC.md`, and hence Phase 7.

Phase 7 — The agentic loop update: giving the loop a logic

DONE

Goal. Supply the reasoning discipline that Phase 6 proved will not emerge on its own — as *structure and code*, not as another voice asking the others to be careful. No new agent.

Why this is one phase and not two

The two defects have one cure. *Absence of evidence read as evidence of absence* and *absence of an instrument read as absence of an effect* are the same sentence with a different subject, and both are fixed by the same move: make the unearned conclusion **unwritable**, and give the loop somewhere to put what it has not yet tried. Splitting them would build the machinery twice.

1. LOGIC.md, and the checker that makes it real

A third source-of-truth document, beside the two that already hold. `ROLES.md` stays honest because `roles.py -check` compares it against the code *in both directions*; `PREMISES.md` stays honest because the Biologist mechanically parses it. `LOGIC.md` states what may be concluded from what, and `logic.py -check` **refuses a malformed claim at the moment the Meta-review writes it** — so a bad claim never enters the document the Proposer reads. A rule agents are merely *asked* to honour is the failure mode being removed.

2. Four modalities, and a home for *could be*

can be	demonstrated positive	one instance suffices
cannot be	universal negative	must carry its quantifier
cannot not be	necessity	the most expensive claim of all
could be	untested, open	free — and today unrepresentable

`memory.md` has exactly two buckets, so untested territory collapses into *impossible*. That is how *we never tried reaction-diffusion* became *the only remaining route is a different geometry*. The third bucket is mineable by the Proposer and may never be cited as a negative.

3. Every negative carries its conditions

Four of seven positive claims carry a falsifier because the template has a **Falsifiable by:** field; two of nine prohibitions carry any escape because it has no field for them. So: a trap, an INERT, a NECESSARY, a *closed* or an *exhausted* records the runs supporting it, the conditions held fixed, and what would refute it — or it does not enter. Necessity is held to the highest bar, because it quantifies over everything not removed. Claims formed before a known apparatus fix are marked *provisional* and queue first for retest.

3b. Converging observations, and what an ablation can carry

A negative needs *converging* support, not repeated support. Rounds 4–8 produced bit-identical results under four different composition hashes, and the morphogen-closure claim cites *ten or more runs* — every one of them Gierer–Meinhardt **with** division, while the one run without it was stable and never counted. Ten runs, **one** observation. A rule demanding “three supporting runs” would have passed the worst claim in the file.

So support is counted in **distinct condition-signatures**, computed from the cited runs’ `composition.json` on disk: strip the claim’s subject, and whatever remains identical across every supporting run is an unvaried confounder. Two consequences come free — the **conditions:** field is *computed rather than authored*, so it cannot be satisfied with prose; and a confounded set is visible at write time instead of five rounds later.

can be	one observation — existence is cheap
could be	none — the default for untried territory
cannot be	three independent signatures, conditions computed
cannot not be	three independent <i>plus</i> one deliberate counterexample attempt that failed

The ablation is the right instrument, read the wrong way. The loop was never short of ablations — `remove_op divide_3d0` is one, and it ran twice, six rounds apart, with

opposite results. The asymmetry is that the *same* experiment supports a positive strongly and a negative weakly: *removing X killed the effect* says X matters *here*; *removing X changed nothing* is offered as X is inert *everywhere*, which is pure absence of evidence. Six INERT claims rest on a single null ablation each — the reading that most needs corroboration got the least. Hence: a **null** ablation must be repeated across independent backgrounds before it may be written as a negative, while a **positive** ablation still passes on one.

And an ablation is *not* a counterexample attempt. Ablation removes X from a working recipe; a counterexample asks whether the effect is reachable *at all* without X, and may change other operators to compensate. Necessity requires the second, which is why “*division is necessary*” survived six rounds — nobody ever tried to build a bud without it. When round 8 finally did, by ablating on a different parent, it produced a clean bud with a fission neck.

Under-supported claims are demoted, never deleted. One null ablation is real evidence; it is simply not a universal negative. It becomes **could be inert** (**n=1, conditions: ...**) — minable by Track A, citable by nobody as a prohibition. The revisit queue is then literally *the list of claims sitting below their support bar*, and the checker’s default action is *demote*, which costs nothing and self-corrects, rather than *reject*, which stalls a round.

4. The tracks become asymmetric

Track A opens. It may violate any trap, and must spend a budgeted quota of slots per round *revisiting* claims — preferring single-run claims and provisional ones. A revisit must vary a **named confounder**, which is what would have caught a family closed on runs that all shared an unvaried one, and what stops the bit-identical duplicates of rounds 4–8. **Track B constrains.** It cites a named Okuda morphology and settings, respects traps, and may not breed from an invalid specimen.

The Proposer’s ablations therefore gain two duties rather than a quota: repeat a **null** ablation on an independent background before it may harden into a negative, and for any necessity claim mount a **rescue attempt** — permitted to change other operators to compensate — because that is the experiment necessity actually rests on.

5. No conclusion about an unmeasured property

If a conclusion needs a quantity no admitted instrument reports, the correct output is a **request for an instrument**, not a verdict. *Sphere* may not be concluded from a run whose pattern was never measured — which is precisely what happened to the finest Turing pattern in the campaign.

6. The Metrologist builds instruments — on demand, not on schedule

The Metrologist gains the power to propose, implement and certify a new instrument against known answers, then admit it. It is **triggered, never scheduled**: the rule above files a request into the Backlog that already exists (`escalation.py`, `operator_requests.jsonl`), and the Metrologist wakes only when the same gap has been requested repeatedly. It runs on its own budget line, outside the round ceiling, because it is not per-round work — expect it to fire perhaps twice in twenty rounds. The cost amortises to nothing: certified once, the instrument is code that runs free forever. The first request is already known: **wavelength, domain count and contrast on the activator field** — the variable that actually governs budding.

7. Paying for it: the ceiling stops binding

Round 10 spent 33.4 minutes against a 25-minute ceiling and began dropping Interpreters. The breakdown says where it goes: **interpreter 4 calls / 13.0 min (39%) and reader 9 calls / 9.8 min (29%)** — two-thirds, both per-run. Cutting effort globally would blunt the roles doing the reasoning, so instead:

- **Lower model effort on the description roles only.** The Reader and the Eye-check *label*; they do not infer. That is the 12.5 minutes that can be halved without touching a judgement.
- **Move the Interpreter earlier in the order.** It runs last, so it is the role truncated — the loop currently pays for the most expensive analysis and then discards it.

Those two alone bring the round under the ceiling, and they buy the headroom Track A's revisit quota will spend.

Exit criteria

- `logic.py -check` reports zero malformed claims, and **rejects the nine-round record** of Phase 6 — every one of the eight errors must be caught by replaying the archived `memory.md` through it.
- A *could be* entry exists, is mined by the Proposer, and is never cited as a negative.
- A Track A slot violates a trap, varies a named confounder, and the result is recorded.
- The Metrologist certifies one new instrument, and a previously-null run is re-read with it.
- A round completes inside the LLM ceiling with no role skipped.

What you get. A loop whose conclusions are as good as its measurements — and which says so when they are not.

Closing Phase 7

What was built

The phase turned out to be wiring, not construction. Almost every mechanism it needed already existed in the codebase and had never run:

- `hypothesis.CLAIM_KINDS` — `sufficient / necessary / causal / descriptive`, with a validator — and all 170 hypotheses in the ledger read `descriptive`, because nothing ever passed the field. A closed set nobody populates is not a type, it is a comment.
- `critic.check_batch` — `A1_NO_ABLATION` already refused a necessity claim with no ablation in the batch, and was named only in `weekend.py` comments.
- `templates.check_memory` — called only under `__main__`; `memory.md` ran 1186 words against a 900-word budget and the check never fired.
- `pattern_scale.py` — wavelength, domain count and spacing on the activator field, **already certified** (*“ n_{spots} exact at 3/5/12, spacing within 13% of $R\sqrt{4\pi/k}$ ”*), computed every frame — and stopped at `metrics`. Not one of the six archived specimens carries a pattern number in the summary an agent reads.

New: `LOGIC.md`, the third source of truth beside `ROLES.md` and `PREMISES.md`; `logic.py` — a claim register, a renderer and the checks, following the `escalation.Backlog` pattern exactly. Support is counted in *distinct condition-signatures* computed from the cited runs’ compositions, so the morphogen-closure claim’s “ten or more runs” — all Gierer–Meinhardt with division — correctly collapses to **one** observation, and `conditions:` is derived from disk rather than authored.

The ablation asymmetry, which was Cedric’s observation and became the load-bearing rule: the loop was never short of ablations. The same experiment supports a positive strongly and a negative weakly — *removing X killed the effect* says X matters here; *removing X changed nothing* was offered as X is inert everywhere. So a **null** ablation must repeat on an independent background, a **positive** one still passes on one, and necessity needs a **rescue attempt** — a different experiment, and the one nobody ever ran.

Budget, since Track A’s revisit quota has to be paid for. Round 10 spent 33.4 minutes against a 25-minute ceiling and dropped Interpreters. `BREVITY` existed, was well written, and reached three prompts out of nine — absent from the two roles that spend the ceiling. It is now applied from `run_agent` with a **per-role token cap and the arithmetic that justifies it** (generation runs at about 70 tokens/s, so 700 tokens is ten seconds). The ceiling drops to ten minutes; projected generation for a round of that shape is 2.5.

What the next round decides — none of this has run

Ten changes rest on a single campaign’s evidence, and not one has executed. `logic.py`’s own rule — three independent signatures before a negative — applies to the conclusions that produced it: *“the Reader’s hardcoded metric list is why agents ignore new instruments”* is a one-observation claim.

So the exit criteria are **stated, not met**, and the next round is the measurement:

change	confirms it	refutes it
token caps, 10-min ceiling	total under 10 min	agents ignore the cap
fast model for the two description roles	reader 1.1 → 0.4 min/call	labels get worse
Interpreter reserves first	no OVER CEILING on it	still dropped
A1_NO_ABLATION live	fires with a sane reason	fires on every batch
claim_kind carried	values besides	all descriptive still descriptive
<code>logic.py</code> per round	unearned count falls	errors, or reports nothing
Metrologist trigger	one or two requests	a flood of operator names
pattern metrics admitted	n_spots_final in diag.json	absent
Reader told they exist	a verdict citing wave-length	never mentioned

Watch A1_NO_ABLATION hardest. It is the only change that can refuse work, it has never executed on a real batch, and it already carried a false-refusal bug that surfaced only when it was finally exercised — **add_op** names the operator and **remove_op** names the node, so the two directions of one experiment never matched and a necessity claim whose ablation sat beside it was refused anyway. It runs in report-only mode for exactly that reason, and starts refusing the round after its first clean pass.

An independent audit of the loop’s information flow — dead ends, orphans, broken **Sends to:** promises — was commissioned rather than done by the author of the changes. It mapped 37 nodes and 65 edges: **26 declared and wired, 16 declared with no code path, 23 wired but undeclared** (`figures/loop_graph.png`). Its sharpest findings are about work committed the same day — **revisits** and **confounder** stored and read by nothing, so the revisit quota is unenforced; `logic_report.jsonl` written with no reader, against its own docstring; A1 promising to refuse “from the round after its first clean pass”, a condition nothing computes. It also found that **track** was declared by the Proposer, printed by the round, and never passed to the constructor — **MISSING on all 58 runs**, so the two-track design had never once been recorded. That one was fixed; the rest wait for data. `ROLES.md` is deliberately untouched.

The false start, 3 August — and what it taught

The first launch reached Act 1 and stopped. Two faults, one of them the phase's own disease surviving its cure.

The ceiling cried wolf. `may_call` added the agent's *timeout allowance* to the round's spend, so with the ceiling lowered to ten it reported "*OVER CEILING at proposer: 0.4+10 > 10.0 min*" on the first call of the round, every round — for a role allowed five minutes that used 1.17. A ceiling that fires before anything is spent is not a ceiling, it is noise, and noise is how a real breach stops being read. It now projects from what the role has *actually* cost this round, then a measured prior, and only then the allowance.

The clean start was not clean. The Proposer emitted twelve slots and the Critic refused eleven as duplicates of the control — zero experiments. The proposal was not even in this campaign's vocabulary: `{"id","spec","override","role"}`, with `spec: embryo_GRO_base.yaml` and `cell_grow.rate=0.6`, from a different project.

The cause was `instruction.md`, which `clean_start` preserved as "*instructions, not state*". Below its `<!-- LEARNED PATTERNS -->` marker sat the round-10 meta-review of the campaign we had just deleted:

"THIS BODY IS CLOSED" "every amplifier family is exhausted"
"STOP proposing ANY 'make the bud bigger' edit on this base"

We wiped `memory.md`, every run and every record — and then handed the first Proposer of the fresh campaign the conclusions we had just declared unsound, with a cold-start frontier and nothing legal left to propose. That block is rewritten by the Meta-review every round, which makes it exactly as much state as `memory.md`. `clean_start` now truncates at the marker, keeping the standing brief and dropping the beliefs, and three further files that had survived it (`batch_refusals`, `campaign.json`, `operator_backlog.md`) are cleared too.

The lesson is the phase's own, turned on itself. An unearned universal negative — *this is closed, that is exhausted* — outlived the deletion of every measurement that produced it, because it was filed as an instruction rather than as a belief. A claim that cannot be checked is bad enough; a claim that cannot be *deleted* is worse.

Phase 8 — Entities, edges and sweeps: the loop stops being a script PROGRESS

IN

Goal. Three things the 3 August runs showed cannot be patched one at a time: give the loop a *declared structure* it can be checked against, close the audit findings that have been open since the graph was drawn, and schedule the parameter sweep that has been implemented and never run.

Why now, and why one phase

Six defects were fixed on 3 August and *not one of them was a logic error*:

<code>composition.json</code>	written by <code>write_config</code> , never by <code>recon</code> : a producer with no consumer
<code>run[:14]</code>	a display truncation read back as an identifier
<code>batch_ok=None</code>	a verdict computed, never applied
<code>menu preamble</code>	a contract asserted in prose, enforced nowhere
<code>max_per_parent=6</code>	a limit set for one purpose binding another
<code>say()</code> vs the wrapper	two owners of one property, folding at 100 and at 96

Every one is an **interface defect**: the components are individually correct and disagree at the seam. That is the class of bug patching cannot retire, because each patch closes one seam and the next round exposes the next. Sixteen more declared edges are unchecked today.

1. Settled by construction: the loop runs with no agent in it

Every defect above was found by *launching*, which is the most expensive instrument available: a round costs ten agent-minutes and half an hour of cluster before it will tell you that a display name was read as an identifier. **The loop must be testable without an agent and without a cluster.** One seam — `run_agent` — carries every model call, and one — `cluster` — carries every job, so both can be substituted for deterministic stubs that return schema-valid answers, including the *wrong* answers we have actually seen: an edit that names a phenotype, a verdict with no flag, a caption with no JSON, a proposal under the wrong key.

A full three-act round then runs offline, in seconds, for nothing, and the questions it answers are exactly the ones that have cost us rounds: does every declared artifact get written, does every gate fire, does the batch fill, is the record complete before its readers run. A regression becomes a failing test rather than a wasted evening. **This is built first, because it is what makes the rest of the phase checkable.**

Three seams, not two. `run_claude` (not `run_agent` — `agents/` is on `sys.path`, so `llm` and `agents.llm` are two module objects for one file, and patching either alone leaves the other live: the first “offline” round spent \$0.37 while printing that it was free), the cluster, and — found only by running — **the VLM**. A 23 GB model load is not offline by any reading, and a harness that takes five minutes will not be run before a launch, which is the only thing it is for.

`offline.py` — a full three-act round in ≈ 6 seconds, no agent, no GPU, no VLM. `test_offline.py` — **8 cases, 8 passing**, each one a round we have already paid for: the prompt arrives whole; no path reaches the real CLI (proved with a `subprocess.Popen` tripwire); a clean round fills its batch and reaches Act 3; phenotype edits are refused and repaired *in the same round*; a proposal under the wrong key is still read; a reply with no JSON does not fabricate a batch; REJECT with no flag is still a rejection; a truncated name resolves to its run.

What it found on its first honest run — the worst defect of the campaign. `run_agent` contained `prompt.split("BREVITY")[0]`, and the Proposer’s template pastes {BREVITY} at character 331 of a 16,093-character prompt. **The agent received 331 characters:** no legal-move menu, no refusals, no history, no output schema. It invented `add branching` and `add chemotaxis` because it had never been shown the operators — and when it reported *“I lack the injected LEGAL MOVES menu (this call omitted it)”*, that was **literally true** and was read here as a hallucination. Four rounds were spent on the consequences, including the recon Proposer naming zero runs, whose JSON schema sat past character 331 as well. Excising the *block* rather than truncating at it: 331 $\rightarrow$ 18,025 characters, 36 moves, schema present.

Two faults were the harness’s own — worth recording, because inventing failures is the characteristic way a fake goes wrong. Inferring the batch size with a loose regex matched “1 slot” inside a pasted refusal block, so it proposed a batch of one; and a hand-written metrics series omitted `act_mean`, so the Diagnostician read all six runs as diverged and stopped the round over an apparatus fault that did not exist. Both now *template from real files* instead of being written by hand, which is the rule this box exists to state: **a fake that invents failures trains you to ignore the alarm.**

Also landed with it. Same-round repair when the Critic guts a batch, because its refusals are mechanical and have a right answer — measured 1 of 12 $\rightarrow$ 12 of 12 on round 2’s real proposal, one extra call, bounded at one so it cannot loop. Attrition written as a number (`asked/proposed/refused/delivered`) rather than two prints a reader must subtract. REVISE reads as a rejection. `[trace]` off by default. And “not checkable” no longer fires on recon slots, which carry no prediction *by design* — it was describing the design as a defect, three lines per run, twelve runs a round.

2. The entities, as classes and a registry

`ROLES.md` already declares who exists and `roles.py -check` keeps it honest — but it checks *existence only*. Each role also declares **Sends to:**, and nothing has ever verified that those fifteen edges correspond to a read anywhere in the code. So a **Role** and an **Artifact** become objects in a registry, each artifact naming its writer, its readers and its schema, and the checker fails the round when:

- an artifact is written and never read — five exist today: `causal_descriptions.md`, `diagnoses.jsonl`, `archivist.jsonl`, `holes.jsonl`, `logic_report.jsonl`

- a declared **Sends to:** edge has no corresponding read
- a prompt asserts a guarantee no checker enforces
- a rendering is assigned to an identifier field

This is the same idiom a fourth time — a document that is the source of truth beside a checker that enforces it — applied to the one thing currently specified only in comments.

3. The audit findings, closed

The round record written after its own consumers; the Archivist's `roll_back` truncating at eleven characters against a twelve-character name; the Eye-check defaulting `supports=True`, so an unparsed caption reads as agreement; recon runs that cannot seed the frontier because a verbatim copy carries no composition; and `_dbg_shrink.py`, which does not import.

4. The sweep that was built and never scheduled

`-mode theta` exists, and its discipline is already mechanical: `comp_hash` excludes `theta`, so a retune cannot register as a new mechanism — a change of numbers cannot masquerade as a new idea because the identity function says so, not because an agent was asked to be careful. What is missing is only wiring: `plan()` emits `recon` and `composition` and never `theta`; nothing chooses the base composition, the parameter or the values; and `build_theta_batch` sweeps *one* parameter of *one* composition, so mixtures of parameters are not expressible at all. Track A is not only “does this mechanism matter” but “what does it do as you turn it up”, and the second question has never been asked.

What you get. A loop whose parts are declared and checked rather than remembered, with the sweep arm switched on. *Nothing relaunched until the checker passes.*

Phase 9 — The demonstration

NOT STARTED

Goal. Run the designed grid — six to eight simulations — that produces the figure: tube, undulation and branching, at Okuda's settings.

What you get. *The figure.*

Phase 10 — The method claim

NOT STARTED

Goal. Write up what the loop found by itself versus what we told it, with the record of predictions and retractions as the evidence.

What you get. The argument that Track A works, resting on Track B.

9. The 31 July lesson, in three lines

Ten defects in one day, every one found by Cedric looking at a picture and none by the loop — a ball that shrank to nothing, a movie where every cell turned green at once, a coral that began uniformly red. They were all the same kind: the loop could tell whether a *simulation* succeeded and had never been able to tell whether the *specimen* was a tissue. Phase 1 is the answer to exactly that, and the only way to find out whether it is enough is to run a batch and see whether the gate catches anything before Cedric does.

10. What I need from you

Nothing blocking. Two things worth knowing:

1. **The buffer gate is the one decision worth taking now.** Phase 5 has already been spent once on runs that could not have produced evidence. A pre-launch refusal costs nothing and would have saved the batch.
2. **Tell me if this note is the right shape.** Too long, too short, wrong order — it is cheap to change, and it is the thing that lets you steer.

Appendix — words I could not avoid

Only these four. Everything else above is ordinary English.

- **Cell sheet (or mesh)** — the model of the tissue: a hollow ball of polygonal cells sharing walls, like a football.
- **Operator** — one mechanism as a reusable building block: “cells divide”, “chemical spreads”, “surface pulls tight”. A model is a combination of these.
- **The loop** — the automated cycle: propose a change, predict the outcome, run, measure, record whether the prediction held.
- **Phase** — one of the eight stages above. Our agreed stopping points.

The four measurements, renamed

Agreed 31 July. The old names misled in three separate ways and are being retired: `protr` reads as a length but is a *ratio*; `final` became ambiguous the moment we introduced the evidence horizon; and `Q` said nothing at all. The rename happens as one mechanical pass once the current work lands, with old records aliased on read — the archive is never rewritten.

was	is	means
<code>protr</code>	<code>elongation</code>	the 95th-percentile cell radius divided by the median. 1.0 is a sphere. A ratio, not a length — reading 1.62 as an amount of protrusion is the mistake the old name invited.
<code>protr_peak</code>	<code>elongation_peak</code>	the most elongated it ever got, over <i>valid</i> frames only.
<code>protr_final</code>	<code>elongation_at_end</code>	at the last <i>valid</i> frame — not simply the last frame.
<code>Q</code>	<code>elongation_unforced</code>	what remains after growth and pushing are switched off and the tissue relaxes. This is the forced-versus-grown test , and it is the whole reason the campaign exists: we can already make a tube by pushing; the question is whether the tissue will hold one by itself.

My working log is `SESSION_LOG.md`; the findings register is `findings.py` and the premises are `PREMISES.md`. You should not need any of them.